

# HBAExpress

## Catalog Service - Cahier des charges technique .NET 9

Document d'implémentation pour Claude Code | ASP.NET Core .NET 9 | PostgreSQL | gRPC | Kafka

**But :** permettre à Claude Code de générer un Catalog Service autonome et production-ready pour la marketplace HBAExpress. Le service gère produits, catégories, marques, attributs, variantes, médias, condition commerciale, révisions, workflow de validation admin et publication vendeur.

Version 1.0 - Août 2026

## 1. Périmètre et responsabilités

Le Catalog Service appartient au domaine Marketplace. Il est propriétaire des données descriptives du produit. Il ne doit pas devenir propriétaire du stock, des commandes, paiements, paniers ou livraisons.

| Fonction                         | Propriétaire                 |
|----------------------------------|------------------------------|
| Produits / révisions             | Catalog Service              |
| Catégories / marques / attributs | Catalog Service              |
| Variantes descriptives           | Catalog Service              |
| Stock / réservations             | Inventory Service            |
| Commandes                        | Marketplace Order Service    |
| Paieement                        | Payment Service              |
| Livraison                        | Delivery Service             |
| Médias binaires                  | Media/Object Storage Service |
| Seller/store status              | Seller Service               |

## 2. Stack technique imposée

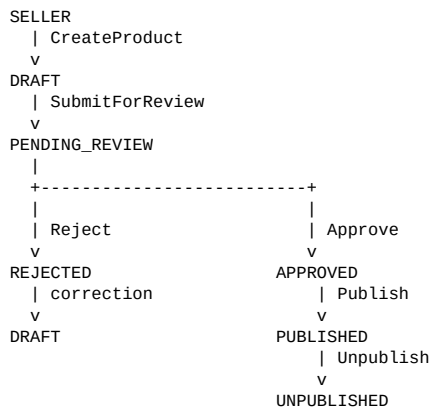
| Composant                | Choix                                     |
|--------------------------|-------------------------------------------|
| Runtime                  | .NET 9                                    |
| Framework                | ASP.NET Core 9                            |
| ORM                      | Entity Framework Core 9                   |
| Base                     | PostgreSQL                                |
| API externes             | REST/JSON                                 |
| Synchrone inter-services | gRPC / Protobuf                           |
| Asynchrone               | Apache Kafka                              |
| Validation               | FluentValidation                          |
| Médiation/CQRS           | MediatR ou handlers maison légers         |
| Mapping                  | Mapster ou mapping explicite              |
| Observabilité            | OpenTelemetry + logs structurés           |
| Tests                    | xUnit + FluentAssertions + Testcontainers |
| Documentation            | OpenAPI/Swagger                           |

## 3. Architecture interne

```
Catalog.Api
|
v
Catalog.Application
|
v
Catalog.Domain
^
|
Catalog.Infrastructure
|- EF Core / PostgreSQL
|- Kafka
|- gRPC clients
|- Outbox / Inbox
|- Cache optional
```

Le domaine ne doit dépendre ni d'ASP.NET Core, ni d'EF Core, ni de Kafka. Les dépendances pointent vers le domaine et l'application.

## 4. Workflow vendeur - admin - publication



ADMIN peut également : PUBLISHED -> SUSPENDED

### Règle absolue

**Un vendeur ne peut jamais publier un produit qui n'a pas été approuvé par un administrateur.** Cette règle doit exister dans l'agrégat domaine, dans le handler applicatif et être couverte par tests E2E. Le frontend ne constitue jamais la barrière de sécurité.

## 5. Statuts produit

| Statut         | Sens                                   |
|----------------|----------------------------------------|
| DRAFT          | Brouillon modifiable par le vendeur    |
| PENDING_REVIEW | Soumis et verrouillé pour validation   |
| APPROVED       | Validé par admin, pas encore publié    |
| REJECTED       | Refusé ; corrections requises          |
| PUBLISHED      | Visible dans la marketplace            |
| UNPUBLISHED    | Retiré volontairement par le vendeur   |
| SUSPENDED      | Bloqué par l'administrateur            |
| ARCHIVED       | Retiré définitivement du cycle courant |

### Transitions autorisées

```
DRAFT -> PENDING_REVIEW
REJECTED -> DRAFT
PENDING_REVIEW -> APPROVED
PENDING_REVIEW -> REJECTED
APPROVED -> PUBLISHED
PUBLISHED -> UNPUBLISHED
UNPUBLISHED -> PUBLISHED
PUBLISHED -> SUSPENDED
APPROVED -> SUSPENDED
SUSPENDED -> APPROVED
DRAFT | REJECTED | UNPUBLISHED -> ARCHIVED
```

## 6. Révisions produit

Une modification critique d'un produit publié ne doit pas écraser immédiatement la version visible. Utiliser ProductRevision. La révision publiée reste servie aux clients pendant qu'une nouvelle révision est en brouillon ou en validation.

```
Product
|- PublishedRevisionId -> rev_004    (visible public)
|- CurrentRevisionId   -> rev_005    (édition vendeur)

rev_005: DRAFT -> PENDING_REVIEW -> APPROVED -> PUBLISHED
```

Modifications critiques : nom, catégorie, marque, description, condition, images principales, variantes, caractéristiques essentielles, type de produit, conformité. Ces changements exigent une nouvelle validation admin.

## 7. Agrégat Product

```
public sealed class Product : AggregateRoot
{
    public Guid Id { get; private set; }
    public Guid SellerId { get; private set; }
    public Guid StoreId { get; private set; }
    public ProductStatus Status { get; private set; }
    public Guid CurrentRevisionId { get; private set; }
    public Guid? PublishedRevisionId { get; private set; }
    public DateTimeOffset? SubmittedAt { get; private set; }
    public DateTimeOffset? ApprovedAt { get; private set; }
    public DateTimeOffset? PublishedAt { get; private set; }
    public DateTimeOffset CreatedAt { get; private set; }
    public DateTimeOffset UpdatedAt { get; private set; }
}
```

## 8. ProductRevision

```
{
  "id": "rev_001",
  "productId": "prod_001",
  "version": 1,
  "name": "iPhone 16 Pro",
  "slug": "iphone-16-pro",
  "shortDescription": "iPhone 16 Pro - 256 Go",
  "description": "Smartphone Apple...",
  "productType": "PHYSICAL",
  "categoryId": "cat_smartphones",
  "brandId": "brand_apple",
  "pricing": {
    "currency": "XOF",
    "basePrice": 850000,
    "compareAtPrice": 900000,
    "costPrice": 760000,
    "taxIncluded": true,
    "taxRate": 18
  }
}
```

## 9. Condition commerciale

La condition commerciale est différente du statut de publication et du stock.

```
public enum ProductConditionType
{
    New,
    OpenBox,
    LikeNew,
    VeryGood,
    Good,
    Fair,
    Refurbished
}
```

### Exemple occasion

```
{
  "type": "VERY_GOOD",
  "grade": "A",
  "description": "Quelques micro-rayures sans impact fonctionnel",
  "isUsed": true,
  "isRefurbished": false,
  "hasOriginalPackaging": false,
  "hasOriginalAccessories": true,
  "functionalStatus": "FULLY_FUNCTIONAL",
  "defects": [
    {
      "type": "COSMETIC",
      "location": "SCREEN",
      "description": "Micro-rayures visibles à contre-jour",
      "severity": "MINOR"
    }
  ]
}
```

### Exemple reconditionné

```
{
  "type": "REFURBISHED",
  "grade": "A",
  "isUsed": true,
  "isRefurbished": true,
  "functionalStatus": "FULLY_FUNCTIONAL",
  "refurbishment": {
    "refurbishedBy": { "type": "PROFESSIONAL", "sellerId": "seller_001" },
    "operations": ["BATTERY_REPLACED", "SCREEN_TESTED", "CAMERA_TESTED"],
    "battery": { "healthPercentage": 94, "replaced": true }
  }
}
```

## 10. Catégories, marques et attributs

Les catégories sont hiérarchiques et les champs du formulaire vendeur sont pilotés par les attributs de catégorie.

```
Electronique
-> Telephones
  -> Smartphones

{
  "categoryId": "cat_smartphones",
  "attributes": [
    { "id": "attr_color", "name": "Couleur", "code": "color", "type": "SELECT", "required": true, "variant": true },
    { "id": "attr_storage", "name": "Stockage", "code": "storage", "type": "SELECT", "required": true, "variant": true },
    { "id": "attr_screen", "name": "Taille écran", "code": "screen_size", "type": "DECIMAL", "unit": "INCH", "required": false }
  ]
}
```

### Types d'attributs

TEXT | TEXTAREA | INTEGER | DECIMAL | BOOLEAN | SELECT | MULTI\_SELECT | COLOR | DATE

Le vendeur ne crée pas directement une nouvelle marque officielle. Si la marque n'existe pas, il crée une BrandRequest validée par un administrateur.

## 11. Variantes

```
{
  "id": "variant_001",
  "sku": "IP16-N256",
  "barcode": "1234567890123",
  "attributes": [
    { "attributeId": "attr_color", "code": "color", "value": "BLACK" },
    { "attributeId": "attr_storage", "code": "storage", "value": "256GB" }
  ],
  "pricing": { "price": 850000, "compareAtPrice": 900000, "currency": "XOF" },
  "isActive": true
}
```

Le SKU est unique. Le stock de la variante est détenu par Inventory Service ; le Catalog Service ne stocke qu'une projection éventuelle de disponibilité pour lecture rapide.

## 12. Médias et spécifications

```
{
  "id": "img_001",
  "url": "https://cdn.hbaexpress.com/products/iphone/main.webp",
  "thumbnailUrl": "https://cdn.hbaexpress.com/products/iphone/thumb.webp",
  "altText": "iPhone 16 Pro noir",
  "displayOrder": 1,
  "isMain": true
}
```

### Contraintes médias

- au moins une image avant soumission
- exactement une image principale
- JPEG/PNG/WebP
- taille maximale configurable
- suppression EXIF recommandée
- scan de sécurité
- stockage binaire hors PostgreSQL

### Spécifications

```
{
  "groups": [
    {
      "name": "Écran",
      "displayOrder": 1,
      "items": [
        { "name": "Type", "value": "Super Retina XDR OLED" },
        { "name": "Taille", "value": "6.3 pouces" }
      ]
    }
  ]
}
```

## 13. Formulaire vendeur - étapes

| Étape | Contenu                                               |
|-------|-------------------------------------------------------|
| 1     | Informations générales : nom, SKU, descriptions, type |
| 2     | Catégorie hiérarchique et marque                      |
| 3     | Condition commerciale / grade / défauts               |
| 4     | Prix / prix barré / coût / taxe                       |
| 5     | Variantes et attributs                                |
| 6     | Stock initial transmis à Inventory Service            |
| 7     | Photos et ordre                                       |
| 8     | Caractéristiques dynamiques de catégorie              |
| 9     | Poids, dimensions, modes livraison, préparation       |
| 10    | Garantie                                              |
| 11    | Aperçu puis enregistrement / soumission               |

## 14. API vendeur

Base : /api/v1/seller/catalog

| Méthode | Route                    | Usage                     |
|---------|--------------------------|---------------------------|
| POST    | /products                | Créer un brouillon        |
| PUT     | /products/{id}           | Modifier produit/révision |
| GET     | /products/{id}           | Détail vendeur            |
| GET     | /products                | Lister ses produits       |
| POST    | /products/{id}/submit    | Soumettre à validation    |
| POST    | /products/{id}/publish   | Publier après approbation |
| POST    | /products/{id}/unpublish | Dépublier                 |
| POST    | /products/{id}/archive   | Archiver                  |
| POST    | /products/{id}/images    | Ajouter média             |
| POST    | /products/{id}/variants  | Ajouter variante          |

### POST /products - exemple

```
{
  "name": "iPhone 16 Pro",
  "shortDescription": "iPhone 16 Pro 256 Go",
  "description": "Smartphone Apple...",
  "productType": "PHYSICAL",
  "categoryId": "cat_smartphones",
  "brandId": "brand_apple",
  "condition": {
    "type": "NEW",
    "hasOriginalPackaging": true,
    "hasOriginalAccessories": true
  },
  "pricing": {
    "basePrice": 850000,
    "compareAtPrice": 900000,
    "currency": "XOF"
  }
}

{
  "success": true,
  "data": {
    "id": "prod_001",
    "status": "DRAFT",
    "currentRevisionId": "rev_001"
  }
}
```



## 15. Soumission et publication vendeur

### POST /products/{id}/submit

Préconditions : vendeur et boutique actifs, nom/description/catégorie/condition/prix valides, image principale, attributs requis, variantes valides et SKU uniques.

```
{
  "success": true,
  "data": {
    "productId": "prod_001",
    "status": "PENDING_REVIEW",
    "submittedAt": "2026-08-18T14:20:00Z"
  }
}
```

### POST /products/{id}/publish

Précondition : status == APPROVED ou UNPUBLISHED et aucune suspension active.

```
{
  "success": false,
  "error": {
    "code": "PRODUCT_NOT_APPROVED",
    "message": "Le produit doit être validé par un administrateur avant publication."
  }
}
```

## 16. API admin

Base : /api/v1/admin/catalog

| Méthode | Route                         | Usage                      |
|---------|-------------------------------|----------------------------|
| GET     | /products/reviews             | File de validation         |
| GET     | /products/{id}/review         | Détail révision            |
| POST    | /products/{id}/approve        | Approuver                  |
| POST    | /products/{id}/reject         | Rejeter avec motifs        |
| POST    | /products/{id}/suspend        | Suspendre                  |
| POST    | /products/{id}/restore        | Restaurer après suspension |
| POST    | /brands/requests/{id}/approve | Valider marque demandée    |

### Rejet exemple

```
{
  "comment": "Le produit nécessite des corrections.",
  "reasons": [
    {
      "code": "INVALID_IMAGES",
      "field": "images",
      "message": "Ajoutez une image principale plus claire."
    }
  ]
}
```

## 17. API publique

Base : /api/v1/catalog. Elle ne doit retourner que la révision publiée des produits PUBLISHED.

| Méthode | Route            | Usage                            |
|---------|------------------|----------------------------------|
| GET     | /products        | Recherche / filtres / pagination |
| GET     | /products/{slug} | Détail produit publié            |
| GET     | /categories      | Arbre catégories                 |
| GET     | /brands          | Marques publiques                |

### Filtres produits

query, categoryId, brandId, sellerId, condition, minPrice, maxPrice, attributes, rating, sort, page, pageSize

### Réponse produit public

```
{
  "success": true,
  "data": {
    "id": "prod_001",
    "name": "iPhone 16 Pro",
    "slug": "iphone-16-pro",
    "condition": { "type": "NEW", "label": "Neuf" },
    "pricing": { "from": 850000, "currency": "XOF" },
    "availability": { "inStock": true, "availableQuantity": 8 },
    "images": [],
    "variants": [],
    "seller": { "id": "seller_001", "storeId": "store_001", "storeName": "HBA Tech Store" }
  }
}
```

**Ne jamais exposer costPrice dans une API publique.**

## 18. Contrats gRPC

```
service SellerService {
  rpc GetSeller(GetSellerRequest) returns (SellerResponse);
  rpc GetStore(GetStoreRequest) returns (StoreResponse);
  rpc ValidateSellerCanSell(ValidateSellerRequest) returns (ValidationResponse);
}

service InventoryService {
  rpc GetAvailability(GetAvailabilityRequest) returns (AvailabilityResponse);
  rpc GetBatchAvailability(GetBatchAvailabilityRequest) returns (BatchAvailabilityResponse);
}

service MediaService {
  rpc ValidateMedia(ValidateMediaRequest) returns (ValidateMediaResponse);
}
```

Tous les appels gRPC doivent avoir deadline/timeout, propagation traceId/correlationId, retry uniquement sur erreurs transitoires et logique idempotente pour les commandes.

## 19. Événements Kafka

Topics recommandés : hba.prod.catalog.product.v1, hba.prod.catalog.product-review.v1, hba.prod.catalog.brand.v1.

### Événements à publier

- catalog.product.created
- catalog.product.submitted
- catalog.product.approved
- catalog.product.rejected
- catalog.product.published
- catalog.product.unpublished
- catalog.product.suspended
- catalog.product.archived
- catalog.brand.requested
- catalog.brand.approved

### Enveloppe

```
{
  "eventId": "0198a43c-...",
  "eventType": "catalog.product.published",
  "eventVersion": 1,
  "occurredAt": "2026-08-18T15:00:00Z",
  "producer": "catalog-service",
  "environment": "production",
  "correlationId": "req_001",
  "causationId": "cmd_001",
  "traceId": "4bf92f...",
  "aggregate": { "type": "Product", "id": "prod_001", "version": 5 },
  "partitionKey": "prod_001",
  "data": {
    "productId": "prod_001",
    "revisionId": "rev_001",
    "sellerId": "seller_001",
    "publishedAt": "2026-08-18T15:00:00Z"
  },
  "metadata": { "schema": "hba.catalog.product.published.v1", "contentType": "application/json" }
}
```

### Fiabilité Kafka

- Transactional Outbox obligatoire
- Inbox/idempotence consumer
- retry avec backoff et jitter
- DLQ après nombre maximal de tentatives
- partitionKey = productId
- eventVersion obligatoire
- correlationId/traceId propagés

## 20. Modèle PostgreSQL

| Table                        | Rôle                               |
|------------------------------|------------------------------------|
| products                     | Agrégat et statut global           |
| product_revisions            | Versions descriptives              |
| product_prices               | Tarification par révision/variante |
| product_conditions           | Condition commerciale              |
| product_condition_defects    | Défauts déclarés                   |
| product_variants             | Variantes                          |
| product_variant_attributes   | Valeurs variantes                  |
| product_images               | Métadonnées médias                 |
| product_specification_groups | Groupes specs                      |
| product_specifications       | Specs                              |
| categories                   | Arbre catégories                   |
| attribute_definitions        | Définitions attributs              |
| category_attributes          | Attributs par catégorie            |
| brands                       | Marques                            |
| brand_requests               | Demandes marques                   |
| product_reviews              | Validations admin                  |
| product_review_reasons       | Motifs de rejet                    |
| outbox_events                | Publication Kafka fiable           |
| consumer_inbox               | Idempotence consumers              |

### products

```
id uuid PK
seller_id uuid NOT NULL
store_id uuid NOT NULL
status varchar(30) NOT NULL
current_revision_id uuid NOT NULL
published_revision_id uuid NULL
created_by uuid NOT NULL
updated_by uuid NOT NULL
created_at timestamptz NOT NULL
updated_at timestamptz NOT NULL
submitted_at timestamptz NULL
approved_at timestamptz NULL
published_at timestamptz NULL
archived_at timestamptz NULL
```

## 21. EF Core - recommandations

Utiliser configurations `EntityTypeConfiguration`, migrations par projet Infrastructure, transactions explicites pour agrégat + Outbox et index adaptés aux requêtes.

```
builder.HasKey(x => x.Id);
builder.Property(x => x.Status).HasConversion<string>().HasMaxLength(30);
builder.HasIndex(x => new { x.SellerId, x.Status });
builder.HasIndex(x => x.PublishedRevisionId);
```

```
// Revision
builder.HasIndex(x => new { x.ProductId, x.Version }).IsUnique();
builder.HasIndex(x => x.Slug);
```

Les montants XOF doivent être stockés en entier (BIGINT/long) pour éviter les erreurs de flottants.

## 22. Permissions / RBAC

| Rôle   | Permissions principales                                                                                        |
|--------|----------------------------------------------------------------------------------------------------------------|
| SELLER | CREATE_PRODUCT, UPDATE_OWN_PRODUCT, SUBMIT_OWN_PRODUCT, PUBLISH_APPROVED_PRODUCT, UNPUBLISH_OWN_PRODUCT        |
| ADMIN  | VIEW_PRODUCT_REVIEW_QUEUE, VIEW_ANY_PRODUCT, APPROVE_PRODUCT, REJECT_PRODUCT, SUSPEND_PRODUCT, RESTORE_PRODUCT |

Toute commande vendeur doit vérifier `SellerId/StoreId` du contexte d'identité contre le propriétaire du Product. Ne jamais accepter `sellerId` librement depuis le body pour autoriser l'accès.

## 23. Validation métier

- name requis et longueur contrôlée
- description requise avant soumission
- category active
- brand active si nécessaire
- condition valide
- basePrice > 0
- currency = XOF par défaut
- au moins une image
- exactement une image principale
- attributs requis présents
- SKU variante unique
- seller actif
- store actif
- pas de publication avant APPROVED

## 24. Codes erreurs standardisés

```
PRODUCT_NOT_FOUND
PRODUCT_NOT_OWNED_BY_SELLER
PRODUCT_ALREADY_SUBMITTED
PRODUCT_NOT_EDITABLE
PRODUCT_NOT_APPROVED
PRODUCT_ALREADY_PUBLISHED
PRODUCT_SUSPENDED
INVALID_PRODUCT_STATUS_TRANSITION
CATEGORY_NOT_FOUND
CATEGORY_INACTIVE
BRAND_NOT_FOUND
BRAND_INACTIVE
INVALID_PRODUCT_CONDITION
INVALID_VARIANT
DUPLICATE_SKU
MISSING_REQUIRED_ATTRIBUTE
INVALID_PRICE
IMAGE_REQUIRED
MAIN_IMAGE_REQUIRED
SELLER_NOT_ALLOWED_TO_SELL
STORE_INACTIVE
PRODUCT_REVIEW_NOT_FOUND
```

### Format erreur

```
{
  "success": false,
  "error": {
    "code": "PRODUCT_NOT_APPROVED",
    "message": "Le produit doit être validé avant publication.",
    "details": { "productId": "prod_001", "currentStatus": "PENDING_REVIEW" },
    "traceId": "4bf92f3577..."
  }
}
```

## 25. Pagination standard

```
{
  "success": true,
  "data": [],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "totalItems": 145,
    "totalPages": 8,
    "hasNext": true,
    "hasPrevious": false
  }
}
```

## 26. Structure solution .NET 9

```
src/
|- HBA.Catalog.Api/
| |- Controllers/
| | |- Seller/
| | |- Admin/
| | `-- Public/
| |- Middleware/
| |- Extensions/
| `-- Program.cs
|
|- HBA.Catalog.Application/
| |- Products/Commands/
| |- Products/Queries/
| |- Categories/
| |- Brands/
| |- Abstractions/
| |- Behaviors/
| `-- Validators/
|
|- HBA.Catalog.Domain/
| |- Products/
| | |- Product.cs
| | |- ProductRevision.cs
| | |- ProductVariant.cs
| | |- ProductReview.cs
| | |- Enums/
| | |- ValueObjects/
| | `-- Events/
| |- Categories/
| |- Brands/
| `-- Common/
|
`-- HBA.Catalog.Infrastructure/
    |- Persistence/
```

```
| | - CatalogDbContext.cs
| | - Configurations/
| | - Migrations/
| | - Repositories/
| - Kafka/
| | - Producers/
| | - Consumers/
| | - Outbox/
| | - Inbox/
| - Grpc/
| | - Clients/
| | - Protos/
| - Observability/

tests/
| - HBA.Catalog.UnitTests/
| - HBA.Catalog.IntegrationTests/
| - HBA.Catalog.ContractTests/
| - HBA.Catalog.E2ETests/
```

## 27. Cas d'usage à implémenter

- CreateProduct
- UpdateProduct
- GetSellerProduct
- ListSellerProducts
- SubmitProductForReview
- ApproveProduct
- RejectProduct
- PublishProduct
- UnpublishProduct
- SuspendProduct
- RestoreSuspendedProduct
- ArchiveProduct
- CreateCategory
- UpdateCategory
- ListCategories
- GetCategoryTree
- CreateBrand
- UpdateBrand
- RequestBrandCreation
- ApproveBrandRequest
- CreateAttributeDefinition
- AssignAttributeToCategory
- AddProductVariant
- UpdateProductVariant
- DeleteProductVariant
- AddProductImage
- DeleteProductImage
- ReorderProductImages
- SetMainProductImage
- GetPublicProduct
- SearchPublicProducts

## 28. Tests obligatoires

### Unit

- transitions de statut
- publication interdite avant approval
- condition commerciale
- prix
- variantes/SKU
- rejet et correction
- suspension/restauration



- création nouvelle révision

## Integration

- EF Core repositories
- migrations PostgreSQL
- Outbox transactionnel
- Inbox/idempotence
- Kafka producer
- gRPC clients avec Testcontainers/mocks appropriés

## E2E principal

1. Seller creates product -> DRAFT
2. Seller completes product
3. Seller submits -> PENDING\_REVIEW
4. Seller calls publish -> 409 PRODUCT\_NOT\_APPROVED
5. Admin approves -> APPROVED
6. Seller publishes -> PUBLISHED
7. Public GET returns the published revision

## E2E rejet

DRAFT -> PENDING\_REVIEW -> REJECTED -> correction -> DRAFT -> PENDING\_REVIEW -> APPROVED -> PUBLISHED

## 29. Observabilité et sécurité

- logs JSON structurés avec Serilog ou provider équivalent
- OpenTelemetry traces REST + gRPC + Kafka
- correlationId propagé
- métriques requêtes, erreurs, latence, Outbox lag
- health checks PostgreSQL/Kafka/gRPC dependencies
- JWT/OIDC via Identity Service
- politiques ASP.NET Core pour SELLER et ADMIN
- ProblemDetails ou enveloppe erreur standardisée
- rate limiting sur endpoints publics
- validation systématique DTO
- aucun secret dans appsettings.json versionné

## 30. Consignes directes pour Claude Code

1. Créer une solution .NET 9 avec 4 projets : Api, Application, Domain, Infrastructure, plus 4 projets de tests.
2. Appliquer Clean Architecture et DDD ; aucune référence framework dans Domain.
3. Utiliser EF Core 9 + PostgreSQL et générer les migrations.
4. Utiliser gRPC pour toutes les communications synchrones inter-services.
5. Utiliser Kafka pour tous les événements asynchrones listés.
6. Implémenter Transactional Outbox et Inbox/idempotence.
7. Implémenter ProductRevision pour protéger la version déjà publiée.
8. Implémenter RBAC SELLER/ADMIN et ownership du vendeur.
9. Bloquer techniquement PublishProduct si la révision n'est pas APPROVED.
10. Créer API Seller, Admin et Public distinctes.
11. Ajouter FluentValidation, OpenAPI, OpenTelemetry et health checks.
12. Écrire tous les tests unitaires, intégration, contrat et E2E décrits.
13. Utiliser UUID v7 lorsque disponible et DateTimeOffset UTC.
14. Ne jamais exposer costPrice dans les DTO publics.
15. Ne jamais utiliser Catalog Service comme source de vérité du stock.

## 31. Critères d'acceptation

Le service est accepté si un vendeur peut créer et enrichir un produit, le soumettre, recevoir un rejet motivé ou une approbation, puis publier uniquement après approbation ; si les révisions publiées restent stables pendant une nouvelle validation ; si les APIs publiques ne montrent que les révisions PUBLISHED ; si Inventory reste source de vérité du stock ; si gRPC/Kafka respectent les choix d'architecture ; et si les tests couvrent le workflow complet.

## 32. Résultat attendu

À la fin de l'implémentation, HBAExpress dispose d'un Catalog Service .NET 9 autonome, versionné, observable, testable et prêt à être intégré aux BFF, Seller Service, Inventory Service, Media Service, Search/Recommandation et aux consommateurs Kafka.